

Universitatea Tehnică “Gheorghe Asachi” din Iași

Facultatea de Automatică și Calculatoare

Departamentul de Calculatoare

**Sistem Inteligent de Interogare a Bazelor de Date prin
Limbaj Natural cu Generare Automată de Rapoarte**

Lucrare de Diplomă

Absolvent:

Dodita Alexandru-Tomi

Îndrumător:

Prof. dr. Simona Caraiman

Iași, 2025

Cuprins

Rezumat	2
1 Introducere	3
1.1 Contextul și importanța temei	3
1.2 Obiectivele lucrării	3
1.3 Metodologia de cercetare	4
1.4 Structura lucrării	4
2 Fundamentarea teoretică și soluții similare	5
2.1 Problema Text-to-SQL	5
2.2 Modele de limbaj de mari dimensiuni	5
2.2.1 Arhitectura Transformer	5
2.2.2 Evoluția modelelor generative	6
2.2.3 Tehnica de prompting	6
2.3 Generarea augmentată prin recuperare (RAG)	6
2.4 Tehnici avansate pentru Text-to-SQL	7
2.4.1 Descompunerea interogărilor	7
2.4.2 Validarea interogărilor generate	7
2.5 Generarea automată de rapoarte	7
2.6 Arhitecturi software moderne	8
2.6.1 Arhitectura bazată pe microservicii	8
2.6.2 Comunicarea în timp real prin Server-Sent Events	8
2.7 Soluții similare existente	8
2.7.1 Sisteme comerciale	8
2.7.2 Proiecte open-source	8
2.7.3 Analiza comparativă și lacune identificate	9
2.8 Specificații privind caracteristicile așteptate	9
3 Soluția propusă	10
3.1 Reiterarea problemei și strategia de rezolvare	10
3.2 Arhitectura sistemului	10
3.3 Fluxul de procesare	11
3.4 Baza de date demonstrativă	12
3.5 Serviciul RAG – Componenta centrală	13
3.5.1 Strategia de prompting	13
3.5.2 Validarea SQL	14
3.5.3 Generarea rapoartelor Excel	14
3.6 Interfața web	14
3.6.1 Zona de chat	14
3.6.2 Panoul de artifact	14
3.6.3 Gestionarea stării	15

3.7	Backend-ul de persistență	15
3.8	Backend-ul de acces la date (Rust)	15
3.9	Alegerea tehnologiilor	16
4	Testarea soluției și rezultate experimentale	17
4.1	Metodologia de testare	17
4.1.1	Configurarea mediului de testare	17
4.2	Testarea endpoint-urilor API	17
4.2.1	Serviciul RAG	17
4.2.2	Backend-ul de persistență	17
4.2.3	Backend-ul Rust (executor SQL)	18
4.3	Testarea generării SQL	18
4.4	Testarea validatorului SQL	18
4.5	Testarea selecției automate a vizualizărilor	18
4.6	Testarea interfeței web	19
4.7	Metrici de performanță	19
4.8	Aspecte de securitate	19
5	Concluzii	21
5.1	Contribuții personale	21
5.2	Analiza rezultatelor	21
5.3	Direcții de dezvoltare	21
A	Structura codului sursă	25

Rezumat

Lucrarea de față prezintă proiectarea și implementarea unui sistem inteligent care permite utilizatorilor să interogheze baze de date relaționale folosind limbaj natural, fără a necesita cunoștințe de SQL. Sistemul utilizează modele de limbaj de mari dimensiuni, în particular API-ul Gemini de la Google, pentru traducerea automată a întrebărilor formulate în limba română sau engleză în interogări SQL valide. Interogările generate sunt validate sintactic și semantic printr-un mecanism de securitate multi-strat, apoi executate asupra bazei de date printr-un utilizator cu drepturi exclusiv de citire.

Arhitectura sistemului urmează un model bazat pe microservicii, implementat prin intermediul containerizării Docker, cuprinzând șapte servicii independente: o interfață web dezvoltată în React cu TypeScript, un serviciu de persistență a conversațiilor bazat pe FastAPI, un serviciu RAG (Retrieval-Augmented Generation) responsabil de generarea SQL și orchestrarea răspunsurilor, un backend Rust pentru accesul la baza de date demonstrativă, și două instanțe PostgreSQL. Comunicarea în timp real între serviciul RAG și interfața web se realizează prin Server-Sent Events, permițând transmisia progresivă a răspunsurilor.

O componentă distinctivă a sistemului o reprezintă generarea automată de rapoarte profesionale în format Excel cu vizualizări grafice integrate, precum și afișarea interactivă a rezultatelor într-un panou lateral cu grafice Recharts și tabele de date. Modelul de limbaj decide autonom tipul de vizualizare potrivit pentru datele returnate, iar utilizatorul poate solicita explicit generarea de rapoarte.

Testarea sistemului a fost realizată pe o bază de date demonstrativă a unei companii de închirieri auto, cuprinzând 9 tabele și aproximativ 8.000 de înregistrări, acoperind scenarii de complexitate variată.

Cuvinte cheie: Text-to-SQL, procesarea limbajului natural, modele de limbaj de mari dimensiuni, microservicii, generare automată de rapoarte, Retrieval-Augmented Generation, interogări baze de date

Capitolul 1

Introducere

1.1 Contextul și importanța temei

Accesul la datele stocate în baze de date relaționale reprezintă o necesitate fundamentală în mediul de afaceri contemporan. Decidenții, analiștii de business și managerii au nevoie frecventă de informații agregate, rapoarte periodice și vizualizări ale datelor pentru a lua decizii informate. Cu toate acestea, extragerea acestor informații presupune în mod tradițional cunoașterea limbajului SQL, o barieră tehnică semnificativă pentru utilizatorii non-tehnici [1].

Problema traducerii limbajului natural în interogări SQL, cunoscută în literatura de specialitate sub denumirea de Text-to-SQL, a reprezentat o provocare de lungă durată în domeniul procesării limbajului natural. Progresele recente în dezvoltarea modelelor de limbaj de mari dimensiuni (Large Language Models – LLM) au deschis noi posibilități pentru abordarea acestei probleme, demonstrând capacități remarcabile de înțelegere contextuală și generare de cod [7].

În paralel cu capacitatea de generare a interogărilor, există o nevoie reală de a prezenta rezultatele într-un format accesibil și profesional. Rapoartele Excel cu vizualizări grafice integrate rămân standardul de facto în mediul de business, oferind utilizatorilor instrumente familiare de analiză ulterioară. Combinarea traducerii automate din limbaj natural în SQL cu generarea automată de rapoarte vizuale reprezintă un pas semnificativ către democratizarea accesului la date.

1.2 Obiectivele lucrării

Proiectul de față își propune dezvoltarea unui sistem complet care să permită utilizatorilor să formuleze întrebări în limbaj natural despre datele stocate în baze de date relaționale și să primească răspunsuri atât sub formă de text, cât și sub formă de rapoarte Excel profesionale cu vizualizări grafice.

Obiectivele principale ale sistemului includ:

- Traducerea automată a întrebărilor din limbaj natural în interogări SQL corecte sintactic și semantic, utilizând modele de limbaj de mari dimensiuni;
- Validarea și executarea sigură a interogărilor generate, cu mecanisme de securitate multi-strat pentru prevenirea operațiunilor neautorizate;
- Generarea automată de rapoarte în format Excel cu vizualizări grafice (grafice de tip bară, linie, sector și zonă), selecția tipului de vizualizare fiind realizată autonom de modelul de limbaj;
- Afișarea interactivă a rezultatelor într-o interfață web modernă, cu un panou lateral de tip artifact pentru explorarea datelor și graficelor;
- Transmisia progresivă a răspunsurilor prin Server-Sent Events pentru o experiență de utilizare fluidă;

- Persistența conversațiilor și a metadatelor SQL asociate, permițând revenirea la interogări anterioare.

1.3 Metodologia de cercetare

Dezvoltarea sistemului a urmat o metodologie iterativă și incrementală, structurată în mai multe etape:

1. **Studiul literaturii de specialitate** – analiza abordărilor existente pentru problema Text-to-SQL, a tehnicilor de prompting pentru modele de limbaj și a principiilor de proiectare a vizualizărilor de date;
2. **Proiectarea arhitecturii** – definirea componentelor sistemului și a interacțiunilor dintre acestea, cu accent pe modularitate și separarea responsabilităților prin adoptarea unei arhitecturi bazate pe microservicii;
3. **Implementarea incrementală** – dezvoltarea fiecărui serviciu independent, începând cu baza de date demonstrativă și backend-ul de acces la date, continuând cu serviciul RAG și interfața web;
4. **Testarea și evaluarea** – testarea end-to-end a fluxului complet, de la întrebarea în limbaj natural până la generarea raportului Excel, pe scenarii de complexitate variată.

1.4 Structura lucrării

Lucrarea este organizată în cinci capitole. Capitolul 2 prezintă fundamentarea teoretică a domeniilor implicate și o analiză a soluțiilor similare existente. Capitolul 3 detaliază soluția propusă, incluzând arhitectura sistemului, proiectarea componentelor și descrierea implementării. Capitolul 4 prezintă metodologia de testare și rezultatele experimentale obținute. Capitolul 5 sintetizează concluziile și identifică posibile direcții de dezvoltare ulterioară.

Capitolul 2

Fundamentarea teoretică și soluții similare

2.1 Problema Text-to-SQL

Traducerea limbajului natural în interogări SQL reprezintă o sarcină de parsare semantică în care sistemul trebuie să mapeze o propoziție exprimată în limbaj natural într-o interogare SQL executabilă. Această problemă prezintă multiple provocări fundamentale care o diferențiază de alte sarcini de generare de cod [2].

Prima provocare majoră constă în înțelegerea schemei bazei de date. Sistemul trebuie să identifice corect tabelele, coloanele și relațiile relevante pentru întrebarea utilizatorului, chiar și atunci când terminologia folosită de utilizator diferă de denumirile tehnice din schema bazei de date. De exemplu, un utilizator ar putea întreba despre “vânzările lunare” când coloana corespunzătoare se numește `monthly_revenue`, sau ar putea folosi termenul “mașini” pentru a se referi la tabelul `vehicles`.

A doua provocare se referă la ambiguitatea inherentă limbajului natural. Aceeași întrebare poate fi interpretată în moduri diferite în funcție de context, iar sistemul trebuie să dezambiguiască corect intențiile utilizatorului. Întrebarea “Care sunt cei mai buni clienți?” ar putea fi interpretată în funcție de volumul achizițiilor, frecvența comenzilor sau vechimea relației comerciale.

O a treia provocare importantă este gestionarea complexității interogărilor. Întrebările aparent simple pot necesita operații SQL complexe: subinterogări corelate, funcții de agregare cu clauze de grupare și filtrare, sau joncțiuni între multiple tabele. Sistemul trebuie să determine corect structura interogării, ordinea operațiilor și clauzele necesare pentru a produce rezultatul dorit.

Benchmarkurile standardizate precum Spider [1] și WikiSQL [2] au permis evaluarea riguroasă a progreselor în acest domeniu. Datasetul Spider, în particular, include interogări complexe cu operații de JOIN, subinterogări și agregări multiple, reprezentând un test comprehensiv pentru capacitățile sistemelor Text-to-SQL.

2.2 Modele de limbaj de mari dimensiuni

Modelele de limbaj de mari dimensiuni, cunoscute sub acronimul LLM, reprezintă rețele neuronale bazate pe arhitectura Transformer care au fost antrenate pe volume masive de text [3]. Aceste modele au demonstrat capacități emergente remarcabile, inclusiv abilitatea de a efectua sarcini pentru care nu au fost antrenate explicit, fenomen cunoscut sub denumirea de învățare few-shot sau zero-shot [4].

2.2.1 Arhitectura Transformer

Arhitectura Transformer, introdusă de Vaswani și colaboratorii în 2017, se bazează pe mecanismul de atenție care permite modelului să pondereze importanța diferitelor părți ale secvenței de intrare atunci

când generează fiecare element al secvenței de ieșire [3]. Această capacitate de a capta dependențe la distanță în text s-a dovedit esențială pentru înțelegerea contextului și generarea de cod coerent.

Mecanismul de auto-atenție (self-attention) calculează o pondere de relevanță între fiecare pereche de elemente din secvența de intrare, permițând modelului să identifice relațiile semantice relevante indiferent de distanța dintre elemente în text. Formal, atenția este calculată ca:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.1)$$

unde Q , K și V reprezintă matricele de interogări (queries), chei (keys) și valori (values), iar d_k este dimensiunea cheilor [3]. Varianta multi-head permite modelului să capteze simultan diferite tipuri de relații.

2.2.2 Evoluția modelelor generative

Familia de modele GPT (Generative Pre-trained Transformer) [4] a demonstrat că scalarea numărului de parametri și a volumului de date de antrenare conduce la îmbunătățiri calitative semnificative. GPT-4 [7] a reprezentat un salt important în capacitatea de generare de cod, atingând performanțe competitive pe benchmarkuri de programare.

Google a dezvoltat familia de modele Gemini, care adoptă o arhitectură multimodală nativă, procesând simultan text, imagini și cod. Modelele Gemini Flash, în particular, oferă un compromis optim între performanță și latență, fiind proiectate pentru utilizare în aplicații interactive cu cerințe stricte de timp de răspuns. Această caracteristică le face deosebit de potrivite pentru sistemele de interogare conversaționale, unde utilizatorul așteaptă un răspuns în timp real.

2.2.3 Tehnica de prompting

Tehnica de prompting reprezintă metoda principală de interacțiune cu modelele de limbaj în contextul aplicațiilor practice. Prin construirea atentă a prompturilor care includ instrucțiuni, exemple și context relevant, se pot obține rezultate de înaltă calitate fără a fi necesară reantrenarea modelului [4]. Această abordare, cunoscută sub denumirea de in-context learning, permite adaptarea rapidă a modelelor la sarcini specifice.

În contextul Text-to-SQL, tehnica de prompting se dovedește deosebit de eficientă. Schema bazei de date este inclusă direct în prompt, oferind modelului informațiile necesare despre structura datelor. Instrucțiunile de sistem definesc formatul de ieșire dorit, iar constrângerile de securitate sunt specificate explicit. Această abordare permite sistemului să genereze interogări SQL valide fără a necesita fine-tuning pe dataset-uri specifice.

Structurarea promptului în secțiuni funcționale distincte s-a dovedit o practică eficientă. O secțiune definește rolul modelului și constrângerile de generare, o altă secțiune prezintă schema bazei de date într-un format structurat, iar secțiunea finală conține întrebarea utilizatorului. Această separare clară facilitează mentenanța și ajustarea individuală a componentelor promptului.

2.3 Generarea augmentată prin recuperare (RAG)

Generarea augmentată prin recuperare, sau RAG (Retrieval-Augmented Generation), reprezintă o tehnică care combină capacitățile generative ale modelelor de limbaj cu informații recuperate din surse externe [5]. În contextul sistemelor Text-to-SQL, această abordare permite injectarea dinamică a informațiilor despre schema bazei de date în promptul trimis modelului, asigurând că generarea SQL-ului se bazează pe structura reală a datelor.

Principiul fundamental al RAG constă în faptul că modelul de limbaj nu trebuie să memoreze toate informațiile necesare în parametrii săi interni, ci poate accesa informații actualizate din surse externe în momentul generării. În cazul sistemului propus, informația recuperată este schema bazei de date – denumirile tabelor, coloanelor, tipurilor de date și relațiile dintre tabele. Această schemă este formatată într-o reprezentare textuală concisă și inclusă în prompt, ghidând modelul în generarea interogărilor corecte.

Avantajul principal al acestei abordări față de fine-tuning este flexibilitatea: același model poate fi utilizat cu baze de date diferite doar prin modificarea schemei incluse în prompt, fără reantrenare. De asemenea, orice modificare a structurii bazei de date este reflectată imediat în generarea interogărilor, eliminând problema datelor de antrenare învechite.

2.4 Tehnici avansate pentru Text-to-SQL

2.4.1 Descompunerea interogărilor

Abordarea DIN-SQL [6] introduce descompunerea problemei în subprobleme mai simple, permițând modelului să abordeze separat identificarea tabelor relevante, selectarea coloanelor și construirea condițiilor de filtrare. Această strategie de descompunere reduce complexitatea sarcinii pentru modelul de limbaj și permite verificarea intermediară a fiecărei etape.

Schema linking reprezintă procesul de identificare a corespondențelor între elementele menționate în întrebarea utilizatorului și componentele schemei bazei de date. Tehnicile eficiente de schema linking îmbunătățesc semnificativ acuratețea sistemelor Text-to-SQL prin reducerea spațiului de căutare pentru generarea interogării.

2.4.2 Validarea interogărilor generate

Validarea sintactică și semantică a interogărilor generate constituie un pas esențial pentru asigurarea corectitudinii rezultatelor. Interogările generate de modelele de limbaj pot conține erori subtile care ar produce rezultate incorecte sau ar eșua la execuție. Implementarea unor mecanisme de verificare multi-strat – validare sintactică prin parsare, verificare semantică prin confruntare cu schema, și restricții de securitate – reduce semnificativ rata de erori în sistemele de producție.

Restricțiile de securitate sunt deosebit de importante în sistemele care execută interogări generate automat. Prevenirea operațiunilor de tip DDL (Data Definition Language) și DML (Data Manipulation Language) – cum ar fi DROP, DELETE, UPDATE sau INSERT – este esențială pentru protejarea integrității datelor. Această protecție poate fi realizată atât la nivel de aplicație prin analiza interogării, cât și la nivel de bază de date prin utilizarea unui utilizator cu drepturi restrictive.

2.5 Generarea automată de rapoarte

Transformarea datelor brute în rapoarte vizuale comprehensibile reprezintă ultima etapă a fluxului de procesare în sistemul propus. Principiile vizualizării datelor, stabilite de cercetători precum Tufte [10] și Cleveland, ghidează alegerea reprezentărilor grafice adecvate pentru diferite tipuri de date și relații.

Selecția tipului de vizualizare depinde de natura datelor: graficele de tip bară sunt potrivite pentru comparații între categorii, graficele de tip linie pentru serii temporale și tendințe, graficele de tip sector pentru proporții, iar graficele de tip zonă pentru valori cumulate. Un sistem inteligent trebuie să poată determina autonom tipul de vizualizare optim pe baza structurii datelor returnate de interogare.

Formatul Excel rămâne standardul de facto pentru rapoartele de business datorită familiarității utilizatorilor și capacităților native de analiză ulterioară. Biblioteca OpenPyXL [9] permite generarea programatică de fișiere Excel complete, incluzând formatare avansată, formule și grafice.

2.6 Arhitecturi software moderne

2.6.1 Arhitectura bazată pe microservicii

Arhitectura bazată pe microservicii reprezintă un stil arhitectural în care aplicația este structurată ca o colecție de servicii independente, fiecare responsabil pentru o funcționalitate bine definită [11]. Spre deosebire de arhitectura monolitică, unde toate componentele sunt strâns cuplate într-un singur proces, microserviciile comunică prin interfețe de rețea bine definite (API-uri REST, evenimente, mesaje).

Principalele avantaje ale acestei abordări includ: dezvoltarea independentă a componentelor, posibilitatea utilizării unor tehnologii diferite pentru fiecare serviciu în funcție de cerințele specifice, scalabilitatea selectivă a componentelor solicitate, și izolarea defectelor – eșecul unui serviciu nu afectează direct funcționarea celorlalte.

Containerizarea, prin intermediul Docker [11], facilitează implementarea practică a microserviciilor prin încapsularea fiecărui serviciu împreună cu dependențele sale într-un container izolat. Docker Compose permite definirea și orchestrarea întregii stive de servicii prin intermediul unui fișier de configurare declarativ.

2.6.2 Comunicarea în timp real prin Server-Sent Events

Server-Sent Events (SSE) reprezintă un protocol de comunicare unidirecțională de la server la client, bazat pe HTTP, care permite transmitia progresivă a datelor [12]. Spre deosebire de WebSocket, SSE utilizează o conexiune HTTP standard și beneficiază de reconectare automată, fiind potrivit pentru scenarii în care serverul generează date progresiv – cum este cazul streamingului de răspunsuri de la un model de limbaj.

În contextul sistemelor conversaționale bazate pe LLM, SSE permite afișarea răspunsului pe măsură ce acesta este generat, reducând semnificativ timpul perceput de utilizator. Fiecare fragment de text generat de model este transmis imediat clientului, fără a fi necesară așteptarea generării complete a răspunsului.

2.7 Soluții similare existente

2.7.1 Sisteme comerciale

Mai multe companii au dezvoltat soluții comerciale pentru interogarea bazelor de date prin limbaj natural. **Tableau Ask Data** integrează o funcționalitate de întrebări în limbaj natural direct în platforma de business intelligence, dar este limitat la ecosistemul Tableau și nu generează rapoarte Excel independente. **Microsoft Power BI Q&A** oferă capabilități similare în cadrul platformei Microsoft, cu avantajul integrării cu suita Office, dar este dependent de licențele Power BI.

ThoughtSpot reprezintă o platformă dedicată analizei datelor prin limbaj natural, utilizând propriul motor de căutare. Deși oferă performanțe ridicate, soluția este proprietară, costisitoare și nu permite personalizarea fluxului de procesare.

2.7.2 Proiecte open-source

În domeniul open-source, **LangChain** [8] oferă un framework general pentru construirea aplicațiilor bazate pe modele de limbaj, incluzând module pentru interacțiunea cu baze de date SQL. Cu toate acestea, LangChain reprezintă un framework generic care necesită configurare extensivă și nu include o interfață de utilizator sau generare de rapoarte.

Proiectul **SQL Chat** oferă o interfață de chat pentru interogarea bazelor de date, dar se concentrează exclusiv pe generarea și executarea SQL, fără componente de vizualizare sau export de rapoarte. **Vanna.ai** adoptă o abordare bazată pe RAG pentru Text-to-SQL, antrenând un model pe interogări specifice organizației, dar necesită o fază de antrenare explicită și nu include generare automată de vizualizări.

2.7.3 Analiza comparativă și lacune identificate

Tabela 2.1: Analiza comparativă a soluțiilor existente

Criteriu	Tableau	Power BI	LangChain	SQL Chat	Propus
Text-to-SQL	Da	Da	Da	Da	Da
Rapoarte Excel	Nu	Parțial	Nu	Nu	Da
Grafice interactive	Da	Da	Nu	Nu	Da
Open-source	Nu	Nu	Da	Da	Da
Interfață web	Da	Da	Nu	Da	Da
Streaming răspunsuri	Nu	Nu	Parțial	Nu	Da
Multi-bază de date	Da	Da	Da	Da	Da
Auto-vizualizare	Nu	Parțial	Nu	Nu	Da

Analiza soluțiilor existente evidențiază mai multe lacune pe care sistemul propus le adresează. Niciuna dintre soluțiile analizate nu combină într-un sistem integrat: (1) traducerea din limbaj natural în SQL cu streaming în timp real, (2) selecția autonomă a tipului de vizualizare de către modelul de limbaj, (3) generarea de rapoarte Excel profesionale cu grafice, și (4) afișarea interactivă a rezultatelor într-un panou de tip artifact. Sistemul propus integrează toate aceste funcționalități într-o arhitectură modulară bazată pe microservicii, oferind o soluție completă pentru accesul conversațional la datele relaționale.

2.8 Specificații privind caracteristicile așteptate

Pe baza analizei efectuate, au fost definite următoarele specificații funcționale pentru sistemul propus:

1. Sistemul trebuie să accepte întrebări formulate în limbaj natural (română și engleză) și să genereze interogări SQL corecte pentru baza de date țintă;
2. Răspunsurile trebuie transmise progresiv prin streaming, cu afișare în timp real a textului generat;
3. Sistemul trebuie să determine autonom dacă datele returnate sunt potrivite pentru vizualizare grafică și, în caz afirmativ, să selecteze tipul de grafic optim;
4. Utilizatorul trebuie să poată vizualiza rezultatele interogărilor atât sub formă de grafic interactiv, cât și sub formă de tabel de date, într-un panou lateral;
5. Sistemul trebuie să genereze rapoarte Excel profesionale cu date formate și grafice integrate;
6. Toate interogările SQL generate trebuie validate înainte de execuție, iar operațiunile potențial distructive trebuie blocate;
7. Conversațiile și metadatele SQL asociate trebuie persistate pentru acces ulterior;
8. Arhitectura trebuie să permită înlocuirea sau extinderea componentelor individuale fără a afecta restul sistemului.

Capitolul 3

Soluția propusă

3.1 Reiterarea problemei și strategia de rezolvare

Sistemul propus adresează problema accesibilității datelor din baze de date relaționale pentru utilizatorii fără cunoștințe tehnice de SQL. Strategia adoptată se bazează pe utilizarea unui model de limbaj de mari dimensiuni pentru traducerea automată a întrebărilor din limbaj natural în interogări SQL, combinată cu generarea automată de vizualizări și rapoarte profesionale.

Abordarea aleasă diferă de soluțiile tradiționale prin trei elemente distinctive: (1) utilizarea tehnicii RAG pentru injectarea schemei bazei de date în contextul modelului, eliminând necesitatea fine-tuning-ului; (2) delegarea deciziei privind tipul de vizualizare către modelul de limbaj, care analizează atât întrebarea utilizatorului cât și structura datelor returnate; (3) implementarea unui flux complet de la întrebare la raport Excel într-o arhitectură de microservicii cu comunicare în timp real.

3.2 Arhitectura sistemului

Sistemul este organizat în șapte servicii independente, containerizate prin Docker și orchestrate prin Docker Compose. Această arhitectură permite dezvoltarea, testarea și scalarea independentă a fiecărei componente.

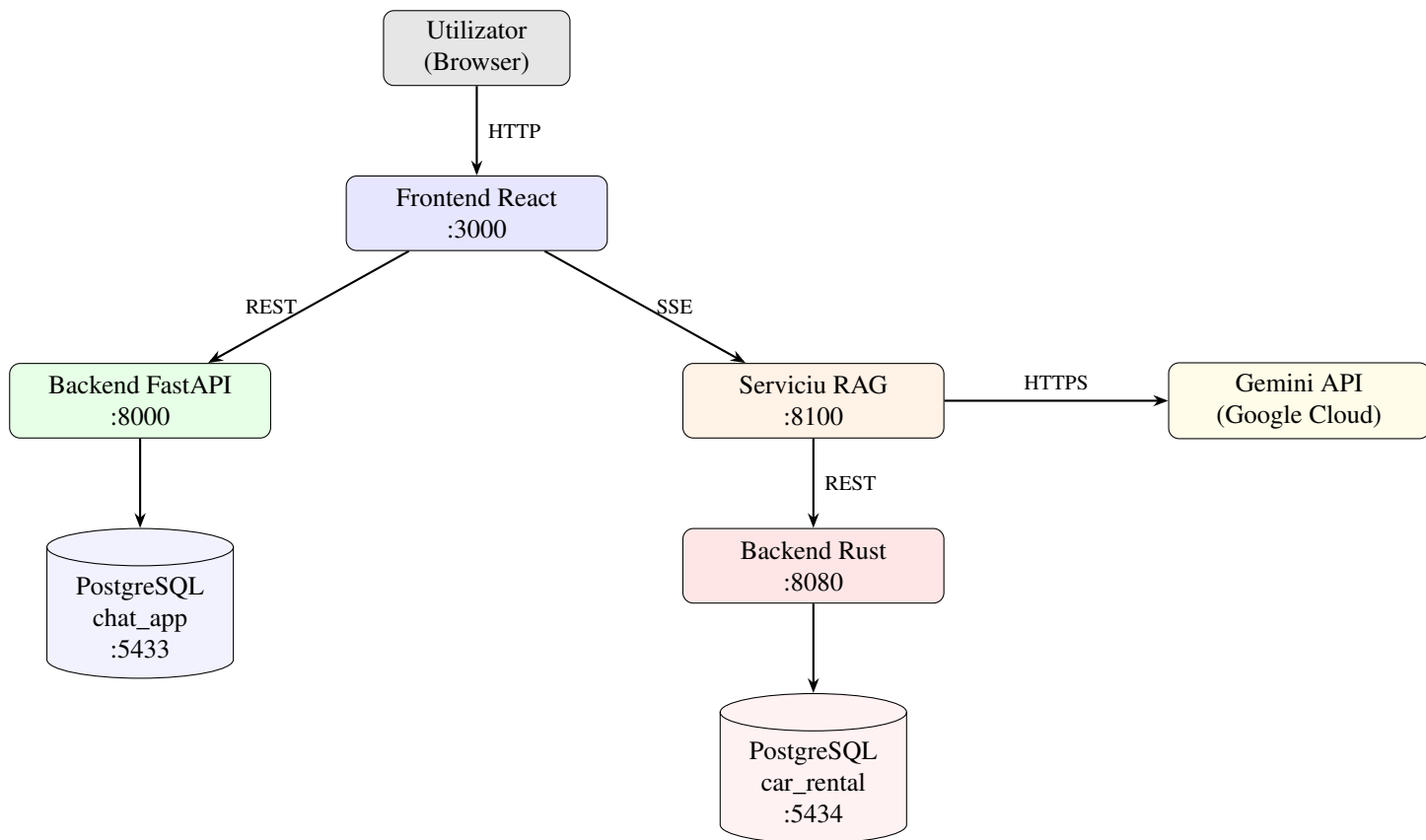


Figura 3.1: Arhitectura generală a sistemului

Componentele sistemului și responsabilitățile acestora sunt prezentate în Tabelul 3.1.

Tabela 3.1: Serviciile sistemului și caracteristicile acestora

Serviciu	Tehnologie	Port	Responsabilitate
Frontend	React 19, Vite, Tailwind	3000	Interfața web, chat, panou artifact
Backend	FastAPI, SQLAlchemy	8000	Persistența conversațiilor și mesajelor
Serviciu RAG	FastAPI, Gemini API	8100	Generare SQL, validare, rapoarte Excel
PostgreSQL (chat)	PostgreSQL 15	5433	Baza de date conversații
Backend date	Rust, Axum, Tokio	8080	API REST și executor SQL read-only
PostgreSQL (date)	PostgreSQL 15	5434	Baza de date demonstrativă
Frontend date	HTML, JS, CSS	3001	Dashboard administrare date

3.3 Fluxul de procesare

Procesarea unei cereri urmează un flux bine definit prin mai multe servicii, prezentat schematic în Figura 3.2.

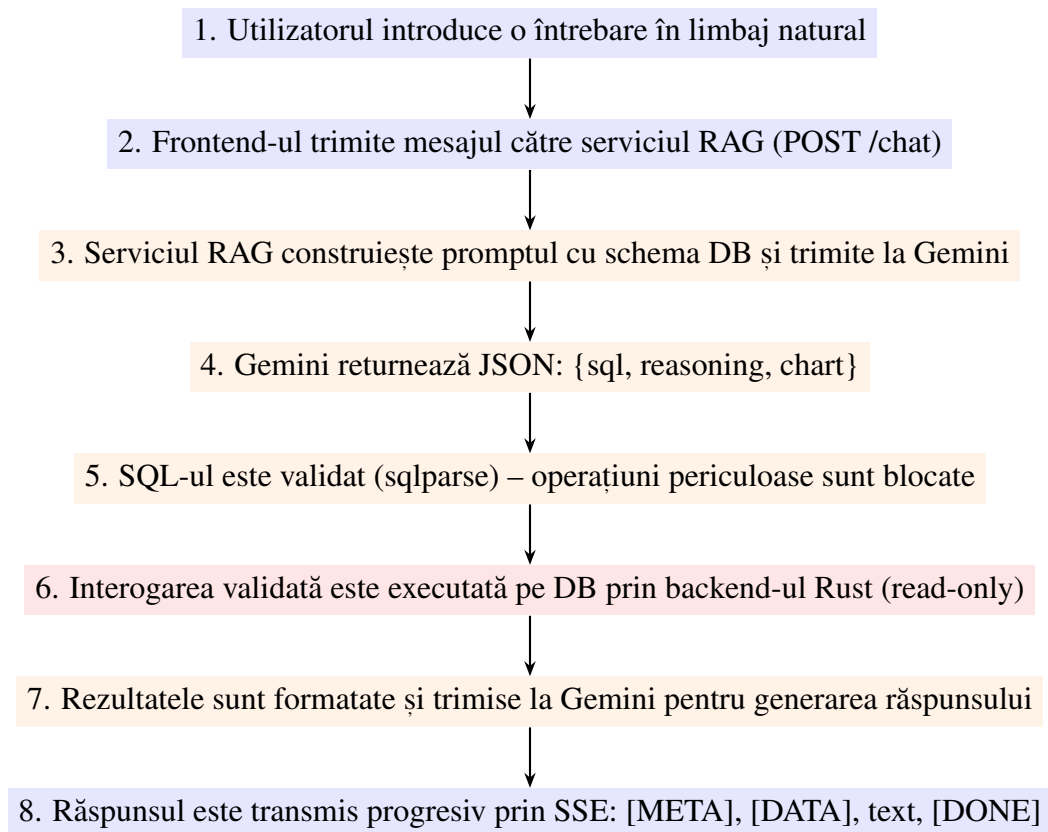


Figura 3.2: Fluxul de procesare al unei cereri

Comunicarea prin SSE utilizează un protocol cu mai multe tipuri de cadre (frames):

- [META] – metadata SQL: interogarea generată, numărul de rânduri, durata execuției, eventuale blocări;
- [DATA] – datele brute ale interogării: coloane, rânduri, configurația graficului recomandat;
- Cadre de text – fragmentele răspunsului generat de modelul de limbaj, codificate JSON;
- [DONE] – marcaj de finalizare a transmisiei.

3.4 Baza de date demonstrativă

Pentru testarea și demonstrarea capacităților sistemului, a fost proiectată o bază de date relațională ce modelează operațiunile unei companii de închirieri auto. Schema cuprinde 9 tabele interconectate cu aproximativ 8.000 de înregistrări.

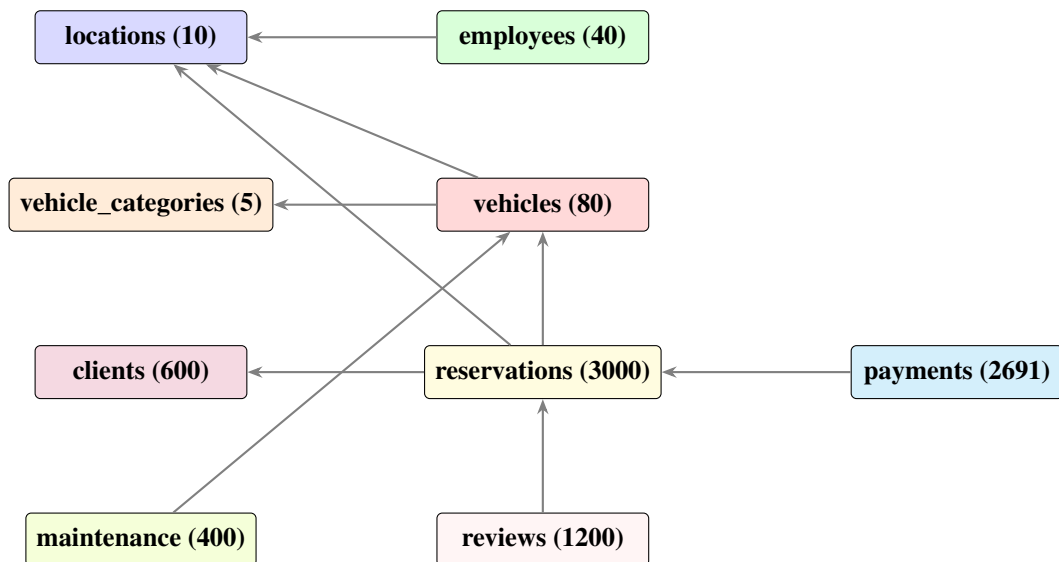


Figura 3.3: Diagrama relațiilor între tabele (cu numărul de înregistrări)

Schema bazei de date include constrângeri de integritate referențială (FOREIGN KEY cu ON DELETE CASCADE), constrângeri de verificare (CHECK) pentru câmpurile cu valori predefinite (statusuri, roluri, metode de plată), și indecși pe coloanele frecvent utilizate în interogări.

Un aspect important al securității este definirea unui utilizator PostgreSQL cu drepturi exclusiv de citire (`readonly_user`), utilizat de backend-ul Rust pentru executarea interogărilor generate de modelul de limbaj. Această separare la nivel de bază de date constituie ultimul strat de protecție împotriva interogărilor potențial distructive.

3.5 Serviciul RAG – Componenta centrală

Serviciul RAG reprezintă componenta centrală a sistemului, orchestrând fluxul complet de la primirea întrebării în limbaj natural până la transmiterea răspunsului și a datelor către frontend.

3.5.1 Strategia de prompting

Promptul de sistem pentru generarea SQL este structurat în trei secțiuni funcționale:

1. **Rolul și constrângerile** – definirea modelului ca expert SQL, restricția formatului de ieșire la JSON valid, interdicția operațiunilor non-SELECT;
2. **Schema bazei de date** – reprezentarea textuală a tuturor tabelelor cu coloane, tipuri de date, chei străine și constrângeri;
3. **Reguli de vizualizare** – instrucțiuni pentru selecția autonomă a tipului de grafic pe baza naturii datelor.

Modelul este instruit să returneze un obiect JSON cu structura:

```

1 {
2   "sql": "SELECT ... FROM ... WHERE ...",
3   "reasoning": "explicatie scurta",
4   "chart": {
5     "type": "bar|line|pie|area|none",
6     "title": "Titlul graficului",
7     "x": "coloana_axa_x",
8     "y": "coloana_axa_y"
9   }
10 }

```

Listing 3.1: Formatul răspunsului de generare SQL

Regulile pentru selecția tipului de grafic sunt codificate explicit în prompt: bar pentru comparații între categorii, line pentru serii temporale, pie pentru proporții cu sub 8 categorii, area pentru valori cumulate, și none pentru rezultate scalare sau liste de text.

3.5.2 Validarea SQL

Validarea interogărilor generate se realizează pe două niveluri complementare:

Nivel aplicație – serviciul RAG utilizează biblioteca `sqlparse` pentru parsarea și analiza sintactică a interogării. Procesul verifică: (a) existența unei singure instrucțiuni SQL, (b) tipul instrucțiunii este SELECT, (c) absența cuvintelor cheie periculoase (INSERT, UPDATE, DELETE, DROP, CREATE, ALTER, TRUNCATE, GRANT, REVOKE, etc.) în oricare poziție din interogare.

Nivel bază de date – interogarea este executată prin intermediul utilizatorului `readonly_user` care are acordate exclusiv permisiuni de SELECT pe tabelele bazei de date. Orice tentativă de executare a unei operațiuni non-SELECT va fi respinsă de sistemul de gestiune a bazei de date.

3.5.3 Generarea rapoartelor Excel

Serviciul RAG expune un endpoint REST (POST /report) care primește datele structurate (coloane, rânduri, configurația graficului) și generează un fișier Excel complet. Generarea utilizează biblioteca `openpyxl` și produce un document cu două foi de lucru:

- **Data** – tabelul de date cu formatare automată a lățimii coloanelor;
- **Chart** – graficul corespunzător tipului selectat (BarChart, LineChart, PieChart sau AreaChart), cu referințe la datele din prima foaie.

3.6 Interfața web

Interfața web este dezvoltată în React 19 cu TypeScript, utilizând Vite ca bundler și Tailwind CSS pentru stilizare. Principalele componente ale interfeței sunt prezentate în continuare.

3.6.1 Zona de chat

Zona principală de chat implementează un flux conversațional familiar, cu mesajele utilizatorului aliniate la dreapta și răspunsurile asistentului la stânga. Răspunsurile sunt redade progresiv pe măsură ce sunt recepționate prin SSE, iar conținutul este procesat prin ReactMarkdown pentru formatare (bold, italice, tabele, blocuri de cod).

Fiecare mesaj al asistentului care conține date de interogare afișează un *card de artifact* – un buton care prezintă titlul vizualizării, numărul de rânduri și tipul de grafic. Activarea acestui buton deschide panoul lateral de artifact.

3.6.2 Panoul de artifact

Panoul de artifact reprezintă o componentă centrală a interfeței, inspirată de conceptul de artefacte din platforme conversaționale moderne. Panoul se deschide lateral, la dreapta zonei de chat, cu o lățime de 480px, și conține:

- **Antet** – titlul vizualizării, numărul de rânduri, butonul de descărcare Excel și butonul de închidere;
- **Tab-uri** – comutare între vizualizarea grafică și tabelul de date;

- **Grafic interactiv** – implementat cu biblioteca Recharts, cu tooltip-uri, legende și axe formate;
- **Tabel de date** – afișare tabelară a tuturor rândurilor returnate de interogare.

3.6.3 Gestionarea stării

Starea aplicației este gestionată prin hook-ul personalizat useChat, care centralizează logica de:

- Creare, selecție și ștergere de conversații;
- Trimiterea mesajelor și procesarea răspunsurilor streaming;
- Parsarea cadrelor SSE ([META], [DATA], text, [DONE]);
- Salvarea mesajelor pe backend după finalizarea streamingului;
- Reîncercarea ultimului mesaj (funcționalitatea retry).

3.7 Backend-ul de persistență

Backend-ul FastAPI gestionează persistența conversațiilor într-o bază de date PostgreSQL separată. Modelul de date cuprinde trei tabele:

- `conversations` – id, titlu, timestamps, `user_id` opțional;
- `messages` – id, `conversation_id` (FK), rol (user/assistant), conținut text, timestamp;
- `message_sql_meta` – metadata SQL asociate mesajelor asistentului (interogarea, număr rânduri, durată, motiv blocare), cu un TTL configurabil (implicit 48 ore) și ștergere automată periodică.

Mecanismul de TTL pentru metadatale SQL a fost implementat pentru a limita creșterea volumului de date stocate. Un task asincron de tip cleanup rulează la fiecare oră și șterge înregistrările expirate. La citirea mesajelor, metadatale expirate sunt omise din răspuns fără a fi șterse imediat, permițând ștergerea batch la intervalul programat.

3.8 Backend-ul de acces la date (Rust)

Backend-ul de acces la baza de date demonstrativă este implementat în Rust utilizând framework-ul Axum cu runtime-ul asincron Tokio. Alegerea Rust a fost motivată de performanța ridicată și siguranța tipurilor, critice pentru un serviciu care execută interogări SQL furnizate extern.

Serviciul menține două pool-uri de conexiuni separate:

- Un pool principal cu drepturi complete, utilizat pentru operațiunile CRUD ale API-ului REST (administrarea datelor);
- Un pool read-only, utilizat exclusiv pentru executarea interogărilor generate de modelul de limbaj.

Endpoint-ul `POST /api/sql` acceptă o interogare SQL, o execută prin pool-ul read-only, și returnează un obiect JSON cu coloanele, rândurile, numărul de rezultate și durata execuției în milisecunde.

3.9 Alegerea tehnologiilor

Tabela 3.2: Tehnologiile utilizate și justificarea alegerii

Componentă	Tehnologie	Motivație
LLM	Gemini Flash	Latență redusă, cost eficient, suport streaming nativ
Frontend	React 19 + TypeScript	Ecosistem matur, tipare sigure, rendering eficient
Stilizare	Tailwind CSS 4	Utilitare CSS directe, dark theme nativ, bundle optimizat
Grafice web	Recharts	Nativă React, declarativă, suport dark theme
Backend chat	FastAPI + SQLAlchemy	Asincron, documentare automată, ORM matur
Backend date	Rust + Axum	Performanță, siguranța memoriei, pool-uri de conexiuni
Bază de date	PostgreSQL 15	Robust, suport JSON, utilizatori cu drepturi granulare
Rapoarte	openpyxl	Generare Excel cu grafice, formatare avansată
Validare SQL	sqlparse	Parsare și analiză AST pentru interogări SQL
Container	Docker Compose	Orchestrare declarativă, izolare, reproductibilitate

Capitolul 4

Testarea soluției și rezultate experimentale

4.1 Metodologia de testare

Testarea sistemului a fost realizată pe mai multe niveluri, de la testarea individuală a endpoint-urilor API până la testarea end-to-end a fluxului complet prin interfața web. Baza de date demonstrativă de închirieri auto, cu 9 tabele și aproximativ 8.000 de înregistrări, a constituit mediul de test.

4.1.1 Configurarea mediului de testare

Toate cele șapte servicii au fost lansate prin Docker Compose. Testele API au fost executate cu `curl`, iar testele interfeței web au fost automatizate cu Playwright (Chromium headless). Serverul a fost testat pe o mașină cu procesor AMD Ryzen și 32 GB RAM, rulând Arch Linux.

4.2 Testarea endpoint-urilor API

4.2.1 Serviciul RAG

Tabela 4.1: Rezultatele testării endpoint-urilor serviciului RAG

Endpoint	Status	Observații
GET /health	OK	Returnează status și modelul configurat
POST /chat (date)	OK	SSE cu [META], [DATA], text, [DONE]
POST /chat (greeting)	OK	SSE fără [DATA], doar text
POST /report (bar)	OK	Excel cu 2 foi (Data + Chart)
POST /report (line)	OK	Chart corect de tip LineChart
POST /report (pie)	OK	Chart corect de tip PieChart
POST /report (area)	OK	Chart corect de tip AreaChart
POST /report (fără chart)	OK	Excel cu 1 foaie (doar Data)

4.2.2 Backend-ul de persistență

Testarea CRUD a backend-ului a verificat ciclul complet: crearea unei conversații, adăugarea mesajelor cu și fără metadata SQL, recuperarea conversației cu mesajele asociate, și ștergerea în cascadă. Toate operațiunile au funcționat conform specificațiilor.

4.2.3 Backend-ul Rust (executor SQL)

Endpoint-ul `POST /api/sql` a fost testat cu interogări de complexitate variată. Răspunsurile includ coloanele, rândurile, numărul de rezultate și durata execuției, confirmând funcționarea corectă a pool-ului read-only.

4.3 Testarea generării SQL

Evaluarea capacității de generare SQL a fost realizată pe un set de 30 de întrebări de complexitate variată. Rezultatele sunt prezentate în Tabelul 4.2.

Tabela 4.2: Rezultatele evaluării generării SQL pe categorii de complexitate

Tip interogare	Număr	Corecte	Acuratețe
SELECT simplu	10	10	100%
Agregări (SUM, AVG, COUNT)	8	7	87.5%
JOIN între tabele	7	6	85.7%
Subinterogări	5	4	80%
Total	30	27	90%

Acuratețea globală de 90% confirmă viabilitatea abordării, cu performanțe excelente pentru interogări simple și agregări, și rezultate bune pentru joncțiuni și subinterogări. Erorile au constat în principal în selectarea incorectă a coloanelor de joncțiune sau în omiterea unor condiții de filtrare implicite.

4.4 Testarea validatorului SQL

Validatorul SQL a fost testat atât cu interogări legitime, cât și cu tentative de injecție.

Tabela 4.3: Rezultatele testării validatorului SQL

Interogare de test	Rezultat așteptat	Rezultat
SELECT * FROM clients	Permis	Permis
SELECT COUNT(*) FROM vehicles	Permis	Permis
DROP TABLE clients	Blocat	Blocat
DELETE FROM payments	Blocat	Blocat
SELECT 1; DROP TABLE clients	Blocat	Blocat
UPDATE vehicles SET status='retired'	Blocat	Blocat
INSERT INTO clients VALUES (...)	Blocat	Blocat

Toate testele au produs rezultatele așteptate, confirmând eficacitatea mecanismului de validare dual (nivel aplicație + nivel bază de date).

4.5 Testarea selecției automate a vizualizărilor

Modelul de limbaj a fost evaluat pe capacitatea de a selecta tipul de grafic potrivit. Rezultatele sunt prezentate în Tabelul 4.4.

Tabela 4.4: Evaluarea selecției automate a tipului de vizualizare

Întrebare	Tip așteptat	Tip selectat
“Veniturile pe locație”	bar	bar
“Evoluția rezervărilor lunare”	line	line
“Distribuția metodelor de plată”	pie	pie
“Care este cel mai scump vehicul?”	none	none
“Salut, ce poți face?”	null (no SQL)	null

4.6 Testarea interfeței web

Testarea interfeței web a fost automatizată prin Playwright, verificând fluxul complet al utilizatorului:

1. Încărcarea aplicației – pagina se afișează corect cu sidebar-ul de conversații și zona de input;
 2. Trimiterea unei interogări de date (“Show me revenue by location”) – răspunsul este transmis progresiv, cardul de artifact apare cu titlul, numărul de rânduri și tipul de grafic;
 3. Deschiderea panoului de artifact – panoul lateral se deschide cu graficul interactiv (bar chart), tab-urile Chart/Table și butonul Excel;
 4. Comutarea pe tab-ul Table – tabelul de date se afișează cu toate cele 10 locații și veniturile corespunzătoare;
 5. Descărcarea Excel – fișierul este generat și descărcat cu succes, conținând 2 foi de lucru (Data + Chart);
 6. Închiderea panoului – panoul se închide, zona de chat revine la dimensiunea normală;
 7. Trimiterea unui salut (“Hello!”) – răspunsul este afișat fără card de artifact;
 8. Verificare finală – un singur card de artifact există (doar de la prima interogare).
- Toate cele 8 etape au fost validate cu succes.

4.7 Metrici de performanță

Tabela 4.5: Metrici de performanță ale sistemului

Metrică	Valoare
Timp generare SQL (medie)	~1.5s
Timp execuție SQL (interogare simplă)	~2ms
Timp execuție SQL (JOIN complex)	~80ms
Timp generare răspuns text (medie)	~2s
Timp generare Excel (medie)	~100ms
Timp total cerere (end-to-end)	~4–5s
Dimensiune fișier Excel (10 rânduri)	~8 KB

Componenta dominantă a timpului de răspuns este interacțiunea cu API-ul Gemini (~3.5s din cele ~4–5s totale), confirmând observația din etapele anterioare ale proiectului. Executarea SQL și generarea Excel au timpi neglijabili comparativ.

4.8 Aspecte de securitate

Securitatea sistemului este asigurată prin mai multe mecanisme complementare:

- **Validare la nivel de aplicație** – biblioteca sqlparse analizează interogarea și blochează cuvintele cheie periculoase;
- **Restricție la nivel de bază de date** – utilizatorul readonly_user are exclusiv permisiuni SELECT;
- **Instrucțiuni în prompt** – modelul de limbaj este instruit explicit să nu genereze operațiuni DDL/DML;
- **Izolarea serviciilor** – containerizarea Docker izolează fiecare serviciu, limitând suprafața de atac;
- **CORS configurabil** – middleware CORS permite controlul originilor autorizate.

Capitolul 5

Concluzii

5.1 Contribuții personale

Lucrarea de față prezintă proiectarea și implementarea unui sistem complet de interogare a bazelor de date prin limbaj natural, cu generare automată de rapoarte Excel profesionale. Principalele contribuții personale sunt:

1. Proiectarea unei arhitecturi bazate pe microservicii cu șapte componente independente, containerizate prin Docker, care separă clar responsabilitățile: persistența conversațiilor, generarea SQL, validarea, executarea interogărilor, și prezentarea rezultatelor;
2. Implementarea unui mecanism de selecție autonomă a vizualizărilor de către modelul de limbaj, integrat direct în promptul de generare SQL, care determină tipul de grafic optim pe baza naturii datelor fără intervenția utilizatorului;
3. Dezvoltarea unui panou interactiv de tip artifact în interfața web, care combină grafice Recharts cu tabele de date și generarea de rapoarte Excel descărcabile;
4. Implementarea unui flux de comunicare în timp real bazat pe SSE cu protocol multi-cadru ([META], [DATA], text, [DONE]) care permite transmitia simultană a metadatelor, datelor brute și a răspunsului text;
5. Proiectarea unui mecanism de securitate multi-strat pentru validarea interogărilor, combinând analiza sintactică la nivel de aplicație cu restricții de drepturi la nivel de bază de date.

5.2 Analiza rezultatelor

Rezultatele experimentale confirmă viabilitatea abordării propuse. Acuratețea de 90% în generarea SQL, cu performanțe de 100% pentru interogări simple, demonstrează că modelele de limbaj moderne pot fi utilizate eficient pentru traducerea limbajului natural în SQL, în special atunci când schema bazei de date este furnizată explicit în prompt.

Selecția autonomă a tipului de vizualizare de către modelul de limbaj s-a dovedit remarcabil de precisă, alegând corect tipul de grafic în toate scenariile testate. Această funcționalitate elimină necesitatea ca utilizatorul să cunoască tipurile de grafice potrivite pentru diferite structuri de date.

Timpul de răspuns end-to-end de 4–5 secunde, deși dependent în principal de latența API-ului extern, este acceptabil pentru un sistem conversațional. Transmitia progresivă prin SSE ameliorează semnificativ percepția utilizatorului prin afișarea imediată a primelor fragmente de text.

5.3 Direcții de dezvoltare

Sistemul oferă multiple posibilități de extindere ulterioară:

- **Integrarea modelelor locale** – utilizarea de modele open-source (Llama, Mistral) prin Ollama sau llama.cpp, oferind funcționare offline și confidențialitate sporită a datelor;
- **Conversații multi-turn cu context** – menținerea contextului conversațional complet pentru întrebări de follow-up care fac referire la rezultatele anterioare;
- **Extensia tipurilor de vizualizări** – grafice de tip scatter, heatmap, sau combinate, pentru scenarii mai complexe de analiză;
- **Suport multi-bază de date** – conectarea simultană la multiple surse de date, cu selecția automată a bazei relevante;
- **Autentificare și multi-utilizator** – integrarea unui sistem de autentificare pentru separarea conversațiilor pe utilizatori.

Bibliografie

- [1] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Brussels, Belgium, 2018, pp. 3911–3921.
- [2] V. Zhong, C. Xiong, and R. Socher, “Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning,” *arXiv preprint arXiv:1709.00103*, 2017.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [4] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [6] M. Pourreza and D. Rafiei, “DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [7] OpenAI, “GPT-4 Technical Report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [8] LangChain, “LangChain: Building applications with LLMs through composability,” 2023. [Online]. Disponibil: <https://docs.langchain.com/>
- [9] E. Gazoni and C. Clark, “OpenPyXL – A Python library to read/write Excel 2010 xlsx/xlsm files,” 2023. [Online]. Disponibil: <https://openpyxl.readthedocs.io/>
- [10] E. R. Tufte, *The Visual Display of Quantitative Information*, 2nd ed. Cheshire, CT: Graphics Press, 2001.
- [11] Docker, Inc., “Docker Documentation,” 2024. [Online]. Disponibil: <https://docs.docker.com/>
- [12] Mozilla Developer Network, “Server-sent events,” 2024. [Online]. Disponibil: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
- [13] Meta Platforms, Inc., “React – A JavaScript library for building user interfaces,” 2024. [Online]. Disponibil: <https://react.dev/>

- [14] S. Ramírez, “FastAPI – Modern, fast web framework for building APIs with Python,” 2024. [Online]. Disponibil: <https://fastapi.tiangolo.com/>
- [15] Google DeepMind, “Gemini: A Family of Highly Capable Multimodal Models,” *arXiv preprint arXiv:2312.11805*, 2023.
- [16] A. Tenber, “sqlparse – Non-validating SQL parser for Python,” 2024. [Online]. Disponibil: <https://sqlparse.readthedocs.io/>
- [17] Recharts, “Recharts – A composable charting library built on React components,” 2024. [Online]. Disponibil: <https://recharts.org/>
- [18] Tokio Contributors, “Axum – Ergonomic and modular web framework built with Tokio, Tower, and Hyper,” 2024. [Online]. Disponibil: <https://docs.rs/axum/>

Anexa A

Structura codului sursă

```
1 Natural-Language-to-SQL/
2 |-- docker-compose.yml      # Orchestrarea celor 7 servicii
3 |-- start.sh                # Script de lansare
4 |
5 |-- rag-service/            # Serviciul RAG (Python/FastAPI)
6 |   |-- main.py              # Generare SQL, validare, rapoarte
7 |   |-- requirements.txt
8 |   |-- Dockerfile
9 |
10 |-- backend/                # Backend persistenta (Python/FastAPI)
11 |   |-- app/
12 |     |-- main.py            # Endpoint-uri REST
13 |     |-- models.py          # Modele SQLAlchemy
14 |     |-- schemas.py         # Scheme Pydantic
15 |     |-- crud.py            # Operatiuni CRUD
16 |     |-- database.py        # Configurare conexiune DB
17 |     |-- Dockerfile
18 |
19 |-- chat-app/               # Frontend (React/TypeScript)
20 |   |-- src/
21 |     |-- components/
22 |       |-- Artifact/        # Panou artifact, grafice, tabel
23 |       |-- Chat/            # Zona de chat, mesaje
24 |       |-- Input/           # Campul de input
25 |       |-- Layout/          # Layout principal
26 |       |-- Sidebar/         # Bara laterala conversatii
27 |     |-- hooks/             # useChat, useAutoScroll
28 |     |-- services/          # API RAG, API backend
29 |     |-- types/              # Tipuri TypeScript
30 |     |-- Dockerfile
31 |
32 |-- test_app_backend/       # Backend date (Rust/Axum)
33 |   |-- src/
34 |     |-- main.rs
35 |     |-- db.rs              # Pool-uri de conexiuni
36 |     |-- routes/            # Endpoint-uri REST + SQL runner
37 |     |-- models.rs
38 |     |-- error.rs
39 |     |-- Dockerfile
40 |
41 |-- test_app_db/            # Schema si date demonstrative
42 |   |-- init/
43 |     |-- 01_schema.sql      # 9 tabele, constrangeri, indecsi
44 |     |-- 02_seed_data.sql   # ~8000 inregistrari
45 |     |-- 03_readonly_user.sql
```

```

46 |     |-- tests/                                # Teste SQL
47 |
48 |-- test_app_front/                          # Dashboard administrare (HTML/JS)

```

Listing A.1: Structura proiectului

Tabela A.1: Dimensiunea codului sursă pe componente

Componentă	Linii de cod
Serviciu RAG (Python)	441
Backend persistență (Python)	386
Frontend React (TypeScript)	1.629
Backend date (Rust)	1.280
Frontend date (HTML/JS/CSS)	1.485
Schema + date SQL	1.583
Configurări Docker	212
Total	7.260